

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке.

Студент гр. 4343

Гизатов Я.Р.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание 1

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

Первая строка – P

Вторая строка – T

Выход:

индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Входные данные

ab

abab

Выходные данные

0,2

Задание 2

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

Первая строка – P

Вторая строка – T

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Входные данные

defabc

abcdef

Выходные данные

3

Задание 1

Выполнение работы

Был реализован алгоритм Кнута-Морриса-Пратта для поиска всех вхождений шаблона P в текст T . Из входного потока считываются две строки: сначала шаблон P , затем текст T . Для пустого шаблона сразу возвращается -1. Построена префикс-функция для шаблона P , после чего выполняется проход по тексту T с использованием значений префикс-функции для эффективного сдвига при несовпадениях. Каждый раз, когда длина совпавшей части становится равной длине шаблона, фиксируется индекс начала вхождения ($i - m + 1$). Если найдено хотя бы одно вхождение, индексы выводятся через запятую, в противном случае выводится -1. Для отладки предусмотрена опция логирования в файл `log1.txt` с детальным выводом всех шагов построения префикс-функции и процесса поиска.

Основные структуры данных

Вектор pi — хранит значения префикс-функции для шаблона P . Длина вектора равна длине P . Каждый элемент $pi[i]$ содержит длину наибольшего собственного префикса подстроки $P[0..i]$, который также является её суффиксом. Тип: `std::vector<int>`.

Вектор `matches` — хранит индексы (позиции) начала всех вхождений шаблона P в текст T . Индексы добавляются последовательно по мере обнаружения совпадений в процессе сканирования текста. Тип: `std::vector<int>`.

Описание алгоритма

Алгоритм Кнута-Морриса-Пратта реализован в виде последовательного пайплайна. На первом этапе из стандартного ввода считываются две строки: сначала шаблон P , затем текст T , после чего проверяется, не пуст ли шаблон. Второй этап — построение префикс-функции для шаблона: создаётся вектор pi размером с длину P , заполняемый

нулями, затем для каждого символа шаблона начиная с индекса 1 вычисляется значение $ri[i]$ на основе предыдущего значения j и сравнения символов $P[i]$ и $P[j]$ с возможными откатами по уже вычисленным значениям ri . Третий этап — поиск вхождений: выполняется проход по каждому символу текста T с поддержанием текущей длины совпадения j , при несовпадении j уменьшается с помощью префикс-функции, при совпадении j увеличивается, а когда j достигает длины шаблона, фиксируется индекс начала вхождения, и j снова корректируется через ri . Четвёртый этап — формирование вывода: если вектор найденных индексов пуст, выводится -1, иначе все индексы выводятся через запятую. Параллельно на всех этапах при активированном флаге логирования выполняется запись подробного отчёта в файл `log1.txt`.

Оценка сложности

Алгоритм Кнута-Морриса-Пратта имеет линейную временную сложность: $O(|P| + |T|)$. Это достигается за счёт того, что построение префикс-функции для шаблона P выполняется за время $O(|P|)$ — каждый символ шаблона сравнивается ограниченное число раз, а переменная j уменьшается не чаще, чем увеличивается, что даёт амортизированную линейность. Аналогично, при поиске по тексту T каждый символ обрабатывается один раз, а откаты по префикс-функции суммарно не превышают длины текста, поэтому фаза поиска занимает $O(|T|)$.

$|P|$ - длина первой строки

$|T|$ - длина второй строки

Оценка по памяти

По памяти алгоритм требует $O(|P| + |T|)$. Основные затраты: хранение входных строк P и T ($O(|P| + |T|)$), массив префикс-функции ri размером $|P|$ ($O(|P|)$), а также вектор `matches` для хранения найденных индексов. В худшем случае (например, текст из одинаковых символов) количество вхождений может достигать $O(|T|)$, но этот вектор является выходными данными, а не вспомогательной памятью, и в задаче его

разрешено возвращать. Без учёта вывода дополнительная память составляет $O(|P|)$.

Задание 2

Выполнение работы

Программа считывает две строки A и B . Проверяется равенство длин; при несовпадении выводится -1 . Для пустых строк выводится 0 . Строится строка $S = A + A$. Вычисляется префикс-функция для строки B . Затем выполняется сканирование S с использованием КМП для поиска B . При обнаружении B в S на позиции pos проверяется, что pos меньше длины A ; если да, то pos выводится как результат. Если pos больше или равно длине A , поиск продолжается до конца S . При отсутствии подходящего вхождения выводится -1 . Логирование этапов выводится в файл `log2.txt`.

Основные структуры данных

Вектор pi — хранит значения префикс-функции для шаблона P . Длина вектора равна длине P . Каждый элемент $pi[i]$ содержит длину наибольшего собственного префикса подстроки $P[0..i]$, который также является её суффиксом. Тип: `std::vector<int>`.

Строка S строится для того, чтобы свести задачу проверки циклического сдвига A относительно B к задаче поиска подстроки B в удвоенной строке A . Действительно, все циклические сдвиги строки A являются её подстроками длины $|A|$ в конкатенации $A + A$, начиная с позиций от 0 до $|A| - 1$. Если B является циклическим сдвигом A , то B обязательно встретится в S на некоторой позиции $pos < |A|$. Если же B не является циклическим сдвигом, то либо B не найдётся в S вовсе, либо найдётся на позиции $pos \geq |A|$ (что соответствует перекрытию через границу и не является допустимым сдвигом). Таким образом, использование S позволяет применить стандартный алгоритм КМП для поиска подстроки и затем проверить условие $pos < |A|$. Тип: `string`.

Описание алгоритма

Реализована программа, которая определяет, является ли строка A циклическим сдвигом строки B . Для этого сначала считываются две строки A и B . Если их длины не равны, сразу выводится -1. Если обе строки пусты, выводится 0 (пустая строка считается сдвигом самой себя). Далее строится строка $S = A + A$ (конкатенация A с собой). Затем вычисляется префикс-функция для строки B . После этого выполняется поиск B в S с использованием КМП. Как только находится вхождение B в S на позиции pos , проверяется, что pos меньше длины A (то есть найденный фрагмент не выходит за пределы первого экземпляра A). В этом случае выводится pos — искомый минимальный сдвиг. Если вхождение найдено, но $pos \geq n$, поиск продолжается. Если ни одного вхождения с $pos < n$ не обнаружено, выводится -1. Всё сопровождается подробным логированием в файл `log2.txt` при установленном флаге `logFlag`.

Оценка сложности

Временная сложность алгоритма составляет $O(|A| + |B|)$. Это достигается за счёт того, что построение строки $S = A + A$ выполняется за $O(|A|)$. Вычисление префикс-функции для строки B требует $O(|B|)$ операций, так как каждый символ B обрабатывается один раз, а суммарное количество уменьшений переменной j в процессе построения pi не превышает $|B|$. Поиск B в S с помощью КМП занимает $O(|S|) = O(2|A|) = O(|A|)$, поскольку алгоритм проходит по каждому символу S ровно один раз, а общее число откатов по префикс-функции в ходе поиска ограничено длиной S .

Оценка памяти

Память, требуемая алгоритмом, оценивается как $O(|A| + |B|)$. Хранятся исходные строки A и B , каждая из которых занимает $O(|A|)$ и $O(|B|)$ соответственно. Для построения $S = A + A$ выделяется новая строка длиной $2|A|$, что даёт $O(|A|)$. Массив префикс-функции pi имеет размер $|B|$, что даёт $O(|B|)$. Дополнительные переменные (счётчики, индексные переменные) требуют $O(1)$ памяти.

Тестирование

Задание 1

Таблица 1 – Тестирование задания 1

№ теста	Входные данные	Выходные данные
Тест 1: Пустой шаблон	hello	-1
Тест 2: Шаблон длиннее текста	abc ab	-1
Тест 3: Несколько непересекающихся вхождений	ab abab	0,2
Тест 4: Пересекающиеся вхождения	aa aaa	0,1
Тест 5: Шаблон не найден	bobr bebra	-1

Задание 2

Таблица 2 – Тестирование задания 2

№ теста	Входные данные	Выходные данные
Тест 1: Пустой шаблон, непустая строка	abc	-1
Тест 2. Сдвиг на 0 (равные строки)	hello hello	0
Тест 3. Строка, не являющаяся сдвигом	asdasd sdsasd	-1
Тест 4. Строки пустые		0
Тест 5. Непустая строка, пустой шаблон	abc	-1

Программы для всех случаев отработала корректно.

Вывод

В ходе выполнения лабораторной работы были реализованы два алгоритма на основе метода Кнута-Морриса-Пратта: для поиска всех вхождений шаблона в текст и для определения, является ли одна строка циклическим сдвигом другой. Оба алгоритма работают за линейное время $O(|P| + |T|)$ и $O(|A| + |B|)$ соответственно, используют $O(|P|)$ и $O(|A| + |B|)$ дополнительной памяти. Проведено тестирование на наборе граничных случаев (пустые строки, строки разной длины, пересекающиеся и непересекающиеся вхождения, циклические сдвиги), результаты соответствуют ожидаемым. Программы корректно обрабатывают входные данные в заданных ограничениях, логирование позволяет отслеживать промежуточные шаги. Таким образом, поставленные задачи решены в полном объёме.

Приложение

Код программы:

1.cpp

```
#include <iostream>
#include <string>
#include <vector>
#include <fstream>

using namespace std;

int main() {

    bool logFlag = true;

    ofstream logFile;
    if (logFlag) {
        logFile.open("log1.txt");
        if (!logFile.is_open()) {
            cerr << "Не удалось создать лог-файл!" << endl;
            return 1;
        }
    }

    string P, T;
    getline(cin, P);
    getline(cin, T);

    if (logFlag) {
        logFile << "Алгоритм Кнута-Морриса-Пратта" << endl;
        logFile << "Шаблон P: \"" << P << "\" (длина " << P.size() <<
        ")" << endl;
        logFile << "Текст T: \"" << T << "\" (длина " << T.size() <<
        ")" << endl << endl;
    }

    int m = (int)P.size();
    if (m == 0) {
        if (logFlag) logFile << "Шаблон пуст, выход с -1." << endl;
        cout << -1 << endl;
        return 0;
    }

    if (logFlag) logFile << "Построение префикс-функции" << endl;
    vector<int> pi(m, 0);
    for (int i = 1; i < m; ++i) {
        int j = pi[i - 1];
        if (logFlag) logFile << "i = " << i << ", j = " << j << ",
        P[i]='" << P[i] << "', P[j]='" << P[j] << "'" << endl;
        while (j > 0 && P[i] != P[j]) {
            if (logFlag) logFile << " несовпадение, уменьшаем j = pi["
            << j-1 << "] = " << pi[j-1] << endl;
            j = pi[j - 1];
        }
        if (P[i] == P[j]) {
            ++j;
            if (logFlag) logFile << " совпадение, новое j = " << j <<
            endl;
        }
        pi[i] = j;
        if (logFlag) logFile << " pi[" << i << "] = " << j << endl;
    }
```

```

    }

    if (logFlag) {
        logFile << "Префикс-функция: [";
        for (int i = 0; i < m; ++i) {
            logFile << pi[i];
            if (i != m-1) logFile << ", ";
        }
        logFile << "]" << endl << endl;
        logFile << "Поиск вхождений" << endl;
    }

    vector<int> matches;
    int j = 0;
    for (int i = 0; i < (int)T.size(); ++i) {
        if (logFlag) logFile << "T[" << i << "] = '" << T[i] << "', j
= " << j << endl;

        while (j > 0 && T[i] != P[j]) {
            if (logFlag) logFile << " несовпадение, переходим j = pi["
<< j-1 << "] = " << pi[j-1] << endl;
            j = pi[j - 1];
        }
        if (T[i] == P[j]) {
            ++j;
            if (logFlag) logFile << " совпадение, j = " << j << endl;
        }

        if (j == m) {
            int start = i - m + 1;
            matches.push_back(start);
            if (logFlag) logFile << " найдено вхождение на позиции "
<< start << endl;
            j = pi[j - 1];
            if (logFlag) logFile << " после обработки j = " << j <<
endl;
        }
    }

    if (logFlag) logFile << "Результат" << endl;
    if (matches.empty()) {
        if (logFlag) logFile << "Шаблон не найден." << endl;
        cout << -1 << endl;
    } else {
        if (logFlag) {
            logFile << "Найденные индексы: ";
            for (size_t k = 0; k < matches.size(); ++k) {
                if (k > 0) logFile << ",";
                logFile << matches[k];
            }
            logFile << endl;
        }
        for (size_t k = 0; k < matches.size(); ++k) {
            if (k > 0) cout << ",";
            cout << matches[k];
        }
        cout << endl;
    }

    if (logFlag) {
        logFile << "Завершение работы." << endl;
        logFile.close();
    }
}

```

```

        return 0;
    }
}

2.cpp

#include <iostream>
#include <string>
#include <vector>
#include <fstream>

using namespace std;

int main() {
    bool logFlag = true;

    ofstream logFile;
    if (logFlag) {
        logFile.open("log2.txt");
        if (!logFile.is_open()) {
            cerr << "Cannot open log file" << endl;
            return 1;
        }
    }

    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string A, B;
    getline(cin, A);
    getline(cin, B);

    if (logFlag) {
        logFile << "Проверка, является ли A циклическим сдвигом B" <<
endl;
        logFile << "A = \"" << A << "\" (длина " << A.size() << ")" <<
endl;
        logFile << "B = \"" << B << "\" (длина " << B.size() << ")" <<
endl << endl;
    }

    int n = A.size();
    int m = B.size();

    if (n != m) {
        if (logFlag) logFile << "Длины не совпадают: " << n << " != "
<< m << " -> -1" << endl;
        cout << -1 << endl;
        return 0;
    }

    if (n == 0) {
        if (logFlag) logFile << "Обе строки пусты -> индекс 0" << endl;
        cout << 0 << endl;
        return 0;
    }

    string S = A + A;
    if (logFlag) logFile << "Строим S = A + A = \"" << S << "\" <<
endl << endl;

    if (logFlag) logFile << "Построение префикс-функции для B" << endl;
    vector<int> pi(m, 0);
    for (int i = 1; i < m; ++i) {
        int j = pi[i - 1];

```

```

        if (logFlag) logFile << "i = " << i << ", символ B[" << i <<
"] = '" << B[i] << "'", начальное j = " << j << endl;
        while (j > 0 && B[i] != B[j]) {
            if (logFlag) logFile << " B[" << i << "] != B[" << j <<
"] (" << B[i] << " != " << B[j]
                << "), уменьшаем j = pi[" << j-1 << "] = " << pi[j-
1] << endl;
            j = pi[j - 1];
        }
        if (B[i] == B[j]) {
            if (logFlag) logFile << " B[" << i << "] == B[" << j <<
"] (" << B[i] << " == " << B[j]
                << "), увеличиваем j до " << j+1 << endl;
            ++j;
        } else {
            if (logFlag) logFile << " j = 0, сравнение не требуется"
<< endl;
        }
        pi[i] = j;
        if (logFlag) logFile << " pi[" << i << "] = " << j << endl <<
endl;
    }

    if (logFlag) {
        logFile << "Префикс-функция pi = [";
        for (int i = 0; i < m; ++i) {
            logFile << pi[i];
            if (i != m-1) logFile << ", ";
        }
        logFile << "]" << endl << endl;
    }

    if (logFlag) logFile << "Поиск B в S (S = A+A)" << endl;
    int j = 0;
    for (int i = 0; i < (int)S.size(); ++i) {
        if (logFlag) logFile << "Шаг " << i << ": S[" << i << "] = '"
<< S[i] << "', текущее j = " << j << endl;
        int step = 0;
        while (j > 0 && S[i] != B[j]) {
            if (logFlag) logFile << " Попытка " << ++step << ": S["
<< i << "] != B[" << j << "] ("
                << S[i] << " != " << B[j] << "), j = pi[" << j-1
<< "] = " << pi[j-1] << endl;
            j = pi[j - 1];
        }
        if (S[i] == B[j]) {
            if (logFlag) logFile << " S[" << i << "] == B[" << j <<
"] (" << S[i] << " == " << B[j]
                << "), увеличиваем j до " << j+1 << endl;
            ++j;
        } else {
            if (logFlag) logFile << " j = 0, символы не сравниваются"
<< endl;
        }
        if (j == m) {
            int pos = i - m + 1;
            if (logFlag) logFile << " Найдено совпадение B в S на
позиции " << pos << " (индексы "
                << pos << "... " << i << ") " << endl;
            if (pos < n) {
                if (logFlag) logFile << " Позиция " << pos << " < n =
" << n << " -> это циклический сдвиг." << endl;
                if (logFlag) logFile << "Результат: " << pos << endl;
                cout << pos << endl;
            }
        }
    }

```

```

        return 0;
    } else {
        if (logFlag) logFile << "    Позиция " << pos << " >= n
= " << n << ", это перекрытие через границу, продолжаем поиск." << endl;
    }
    if (logFlag) logFile << "    Обновляем j = pi[" << m-1 << "]"
= " << pi[m-1] << endl;
    j = pi[j - 1];
    }
    if (logFlag) logFile << "    Новое j = " << j << endl << endl;
}

    if (logFlag) logFile << "Совпадение не найдено в пределах первых n
символов S -> -1" << endl;
    cout << -1 << endl;
    return 0;
}

```